# 10. Random Forest

**Why one tree when we can train many trees?**

**Previous Section:**

9. Decision Trees

**Contents:**



This section is about Random Forest—and to understand why it exists, you need to remember what we already know about Decision Trees. They are powerful. In fact, they can become *too* powerful.

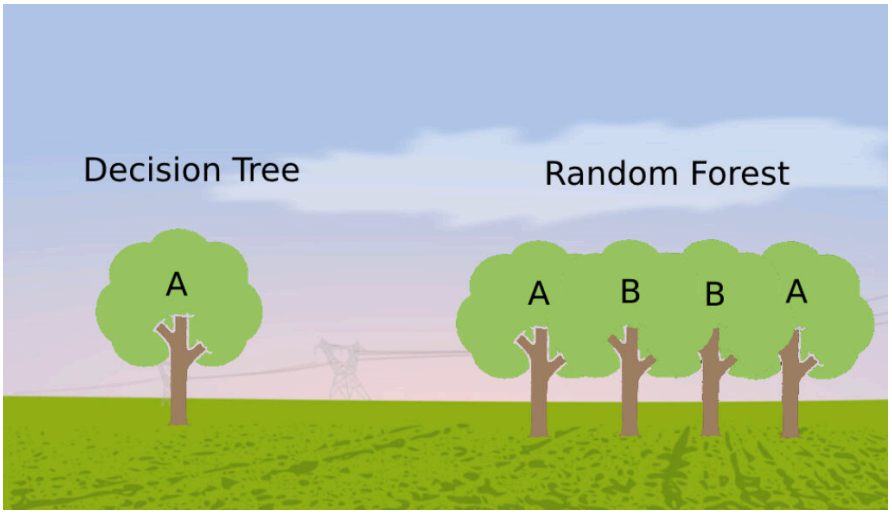
A Decision Tree can split data very quickly, sometimes using just two or three features out of ten or twelve to perfectly separate the training data. And that sounds impressive... until you realize what's actually happening.

It means the model is starting to memorize rather than understand. This is what we call overfitting—the tree becomes too specific to the training data, too "**predictable**" in a way that doesn't generalize well to new, unseen data. It learns the noise along with the signal.

So now the question becomes: **What if we don't rely on just one tree?**

What if instead of trusting a single model that might overfit, we train many trees—and let them collectively decide?  That's where the idea of a "forest" comes from. Many trees, working together.

Check out this image:




At first, you might think the process is simple:

> ***Take your dataset, train Tree 1, then Tree 2, then Tree 3... all the way up to Tree 100.***

And technically that's what happens.  But there's a problem hidden inside that idea.

If every tree is trained on the exact same dataset, using the same method like Information Gain, then almost all of those trees will end up looking absurdly identical. And if they are identical, then having 100 trees is no better than having just one.

That's why the word **"Random"** in Random Forest is not just a name—it's the key idea.

Instead of training every tree on the full dataset, we train each tree on a **random subset** of the data. These subsets are created by randomly sampling from the original dataset, meaning each tree sees a slightly different version of the world.

And because each tree learns from different data, they grow differently, make different mistakes, and capture different patterns. That is where the real power begins.

## 1. Random Forest – Training many decision trees at the same time

Now that we understand the intuition, let's formalize it a bit:

> Random Forest, by definition, is a Machine Learning model that extends from Decision Trees —but instead of training a single tree, it trains **multiple decision trees at the same time**, all using the same underlying algorithm.

The idea is simple: *don't rely on one model, rely on many.*

The term "*Forest"* already tells you something important—it means more than one tree. But these trees are not identical.

- Each tree is different because of the randomness involved.
- Instead of seeing the entire dataset, every tree is trained on a **different subset of the data**, sampled from the original dataset.

- So even though they all use the same learning process, they grow in different ways and capture different patterns.

This brings us to an important concept:

> Random Forest is an **ensemble learning method**. That simply means we train multiple models and then combine their results to make a final decision. Instead of trusting a single perspective, we aggregate many perspectives.

More specifically, Random Forest belongs to a technique called **Bagging** (Bootstrap Aggregating). This means:

- Each tree is trained **individually and independently**.
- There is no communication between the trees during training—each one learns on its own, based only on the subset of data it receives.
- In other words, every model forms its own understanding of the problem without being influenced by the others.

A simple way to think about this is to imagine a group of students solving the same problem:

- But instead of giving each student the full question, you give each of them only a **part of the information**.
- Naturally, their answers will differ slightly. Some might focus on certain aspects, others on different details—but together, they provide a more balanced and reliable answer than any single student alone.

So after all the trees make their predictions, how do we combine them into one final result?

- For **classification**, we use **majority voting**—the class that gets the most votes from the trees becomes the final prediction.
- For **regression**, we take the **average** of all the predictions.

And that's the core idea of Random Forest:

> Many independent models, each slightly different, coming together to produce a stronger and more reliable result than any single tree could achieve on its own.

## 2. Random Forest – Random Sampling with Replacement

Now we move to the core mechanism behind Random Forest—**Random Sampling with Replacement**, also known as **bootstrapping**.

Sampling with replacement is a technique where we create a subset of the dataset by randomly selecting samples, but with one important twist:

> **The same sample is allowed to appear multiple times in the same subset**.

Think about that carefully. When we pick a row from the dataset, we don't remove it. It goes back into the pool, meaning it can be selected again. And because of that, a single subset might contain duplicate rows, while some original rows might not appear at all.

This solves an important problem.

- If we were to sample *without* replacement, each subset would quickly become too small or too similar—especially when working with limited data.
- But with replacement, we can create many subsets, each with a reasonable size, even if the original dataset isn't very large. In fact, each subset can be as large as, or even conceptually larger than the original dataset, simply because duplicates are allowed.

And of course, the word **"Random"** means exactly what you think:

> Every sample is selected **randomly**. No pattern, no ordering—just pure stochastic selection.

So what does this actually look like?

You are given the dataset below (14 samples):

| # | Age | Income | Student | Credit_Rating | Buy_XBOX |
|---|---|---|---|---|---|
| 1 | ≤30 | high | no | Fair | no |
| 2 | ≤30 | high | no | Excellent | no |
| 3 | 31…40 | high | no | Fair | yes |
| 4 | >40 | medium | no | Fair | yes |
| 5 | >40 | low | yes | Fair | yes |
| 6 | >40 | low | yes | Excellent | no |
| 7 | 31…40 | low | yes | Excellent | yes |
| 8 | ≤30 | medium | no | Fair | no |
| 9 | ≤30 | low | yes | Fair | yes |
| 10 | >40 | medium | yes | Fair | yes |
| 11 | ≤30 | medium | yes | Excellent | yes |
| 12 | 31…40 | medium | no | Excellent | yes |
| 13 | 31…40 | high | yes | Fair | yes |
| 14 | >40 | medium | no | Excellent | no |

Now suppose we want to create **three subsets**, and each subset contains **10 samples**. Using sampling with replacement, we might get something like this:

**Subset 1 (10 samples):**

| # | Age | Income | Student | Credit_Rating | Buy_XBOX |
|---|---|---|---|---|---|
| 1 | ≤30 | high | no | Excellent | no |
| 2 | >40 | low | yes | Fair | yes |
| 3 | >40 | low | yes | Fair | yes |
| 4 | ≤30 | low | yes | Fair | yes |
| 5 | ≤30 | high | no | Fair | no |
| 6 | 31…40 | medium | no | Excellent | yes |
| 7 | ≤30 | high | no | Excellent | no |

| # | Age | Income | Student | Credit_Rating | Buy_XBOX |
|---|---|---|---|---|---|
| 8 | >40 | medium | no | Excellent | no |
| 9 | 31…40 | low | yes | Excellent | yes |
| 10 | 31…40 | high | no | Fair | yes |

**Subset 2 (10 samples)**

| # | Age | Income | Student | Credit_Rating | Buy_XBOX |
|---|---|---|---|---|---|
| 1 | >40 | medium | no | Fair | yes |
| 2 | ≤30 | medium | no | Fair | no |
| 3 | ≤30 | medium | no | Fair | no |
| 4 | >40 | medium | yes | Fair | yes |
| 5 | >40 | low | yes | Excellent | no |
| 6 | ≤30 | medium | yes | Excellent | yes |
| 7 | 31…40 | high | yes | Fair | yes |
| 8 | >40 | medium | no | Fair | yes |
| 9 | ≤30 | high | no | Excellent | no |
| 10 | ≤30 | low | yes | Fair | yes |

Subset 3 (10 samples)

| # | Age | Income | Student | Credit_Rating | Buy_XBOX |
|---|---|---|---|---|---|
| 1 | ≤30 | high | no | Fair | no |
| 2 | 31…40 | high | no | Fair | yes |
| 3 | 31…40 | high | no | Fair | yes |
| 4 | >40 | low | yes | Fair | yes |
| 5 | >40 | low | yes | Excellent | no |
| 6 | >40 | low | yes | Excellent | no |
| 7 | 31…40 | medium | no | Excellent | yes |
| 8 | >40 | medium | yes | Fair | yes |
| 9 | >40 | medium | no | Excellent | no |
| 10 | 31…40 | low | yes | Excellent | yes |

Notice a few important things:

- Some rows appear multiple times within the same subset.
- Some rows from the original dataset are completely missing in a subset.
- Every subset is different, even though they all come from the same original data.

And this is exactly what we want. Because now, when each Decision Tree is trained on its own subset, it sees a slightly different version of reality.

This leads to different splits, different structures, and ultimately, different predictions.

And when we combine all those trees together, we reduce overfitting, increase robustness, and build a model that is far more reliable than a single Decision Tree.

That is the true power of randomness.

## 3. Random Forest – Training Process

Now let's connect everything back to what you already know.

Before Random Forest, we studied Decision Trees—specifically the **ID3 algorithm**, where we used **Entropy** and **Information Gain (IG)** to decide the best feature for splitting. In your Jupyter Notebook, you already implemented this step by step: computing entropy, calculating IG for each feature, and selecting the best one.

Now, in Random Forest, nothing changes at the core. Each tree still uses the **exact same Decision Tree algorithm**.

The difference is not *how* a tree is trained—but *what data it sees*.

So the process becomes:

- First, we take one of the sub-datasets created using **random sampling with replacement**. This subset becomes the training data for a single tree.

Then, we apply the same steps you already know:

- Compute the **entropy** of the dataset
- Compute the **Information Gain** for each feature
- Select the **best feature** (the one with highest IG)
- Split the dataset based on that feature
- Repeat the process recursively for each subset

In other words, we reconstruct the entire Decision Tree pipeline—but now it runs **independently for each sub-dataset**.

Here, we let our little friend **pandas** to help us with the calculations.

```
from math import log2
import pandas as pd
```

- A function to compute **entropy**

```
# Entropy of target
def entropy_target(target):
    p_list = []
    for val in target.unique():
        p = target.value_counts().get(val, 0) / len(target)
        p_list.append(p)
    return sum(-p * log2(p) for p in p_list if p > 0)
```

```
# Entropy of feature
def entropy_feature(feature, target):
    df = pd.concat([feature, target], axis=1)
    entropy_list = []

    for f_val in feature.unique():
        p_list = []
        num_class = feature.value_counts().get(f_val, 0)

        for t_val in target.unique():
            sub_df = df[(df[feature.name] == f_val) &
                        (df[target.name] == t_val)]
            p = len(sub_df) / num_class if num_class != 0 else 0
            p_list.append(p)

        entropy = sum(-p * log2(p) for p in p_list if p > 0)
        entropy_list.append(entropy)

    return entropy_list
```

- A function to compute **Information Gain**

```
# Information Gain
def information_gain(x, y):
    e_target = entropy_target(y)
    results = {}

    for col in x.columns:
        feature = x[col]
        unique_counts = [feature.value_counts().get(v, 0) for v in feat
ure.unique()]
        entropy_list = entropy_feature(feature, y)

        # Weighted entropy
        weight = sum((unique_counts[i] / len(x)) * entropy_list[i]
                     for i in range(len(unique_counts)))

        ig = e_target - weight
        results[col] = {"information_gain": ig}

    best_feature = max(results, key=lambda k: results[k]["information_g
ain"])

    return results, best_feature
```

- Recursive Splitting

```
def recursive_split(X, y, best_feature):
    next_best = {}

    for value in X[best_feature].unique():
        print(f"\nSubset: {best_feature} = {value}")

        sub_X_full = X[X[best_feature] == value]
        sub_y = y[X[best_feature] == value]

        # Concatenate
        print(pd.concat([sub_X_full.drop(columns=[best_feature]), sub_
y], axis=1))

        # If pure → store leaf
        if len(sub_y.unique()) == 1:
            print("Pure subset")
            next_best[value] = {"type": "leaf", "class": sub_y.iloc[0]}
            continue

        # Otherwise compute next best feature
        sub_X = sub_X_full.drop(columns=[best_feature])
        sub_info, sub_best_feature = information_gain(sub_X, sub_y)
        for feature, info in sub_info.items():
            print(f"{feature}: {info}")
        print("+ Best Feature:", sub_best_feature)

        next_best[value] = {"type": "node",
                            "information_gain": sub_info,
                            "value": value,
                            "best_feature": sub_best_feature}

    return next_best
```

```
# Now we filter the dataset based on the root feature and the best feat
ure and check for its impurity
def process_next_level(X, y, best_feature, next_best_features):
    all_next_levels = {}
    for value, info in next_best_features.items():
        print(f"\nProcessing branch: {best_feature} = {value}")
        # Only continue if impure
        if info["type"] == "node":
            sub_best_feature = info["best_feature"]
            # Filter using root feature
```

```
            X_sub = X[X[best_feature] == value].drop(columns=[best_feat
ure])
            y_sub = y[X[best_feature] == value]
            print(f"→ Going deeper using feature: {sub_best_feature}")
            # Call recursive_split again
            next_level = recursive_split(X_sub, y_sub, sub_best_featur
e)
            # Store result (optional but useful)
            all_next_levels[value] = next_level
        else:
            print("→ Leaf node, stopping.")
            all_next_levels[value] = info

    return all_next_levels
```

And now, instead of calling this process once, we call it **multiple times**, once for each randomly generated subset.

- For Sub-Dataset 1

```
sub1 = {
    "Age": ["≤30", ">40", ">40", "≤30", "≤30", "31…40", "≤30", ">40",
"31…40", "31…40"],
    "Income": ["high", "low", "low", "low", "high", "medium", "high",
"medium", "low", "high"],
    "Student": ["no", "yes", "yes", "yes", "no", "no", "no", "no", "ye
s", "no"],
    "Credit_Rating": ["Excellent", "Fair", "Fair", "Fair", "Fair", "Exc
ellent", "Excellent", "Excellent", "Excellent", "Fair"],
    "Buy_XBOX": ["no", "yes", "yes", "yes", "no", "yes", "no", "no", "y
es", "yes"]
}

X = pd.DataFrame(sub1)
y = X.pop("Buy_XBOX")
results, best_feature = information_gain(X, y)

# Iterate through the result dictionary
for feature, info in results.items():
    print(f"{feature}: {info}")
print("+ Best Feature:", best_feature)
```

Age: {'information_gain': np.float64(0.3709505944546686)}
Income: {'information_gain': np.float64(0.4464393446710154)}
Student: {'information_gain': np.float64(0.4199730940219749)}
Credit_Rating: {'information_gain': np.float64(0.12451124978365313)}
+ Best Feature: Income

```
next_best_features = recursive_split(X, y, best_feature)
```

```
    Age Income Student Credit_Rating Buy_XBOX
0   ≤30   high      no     Excellent       no
4   ≤30   high      no          Fair       no
6   ≤30   high      no     Excellent       no
9  31…40   high      no          Fair      yes
Age: {'information_gain': 0.8112781244591328}
Student: {'information_gain': 0.0}
Credit_Rating: {'information_gain': 0.31127812445913283}
+ Best Feature: Age

    Age Income Student Credit_Rating Buy_XBOX
1   >40    low     yes          Fair      yes
2   >40    low     yes          Fair      yes
3   ≤30    low     yes          Fair      yes
8  31…40    low     yes     Excellent      yes
This subset is already pure. Skipping...

    Age  Income Student Credit_Rating Buy_XBOX
5  31…40  medium      no     Excellent      yes
7   >40  medium      no     Excellent       no
Age: {'information_gain': 1.0}
Student: {'information_gain': 0.0}
Credit_Rating: {'information_gain': 0.0}
+ Best Feature: Age
```

```
all_next_levels = process_next_level(X, y, best_feature, next_best_features)
```

Processing branch: Income = high
→ Going deeper using feature: Age

Subset: Age = ≤30
Student Credit_Rating Buy_XBOX
0 no Excellent no
4 no Fair no
6 no Excellent no
Pure subset

Subset: Age = 31...40
Student Credit_Rating Buy_XBOX
9 no Fair yes
Pure subset

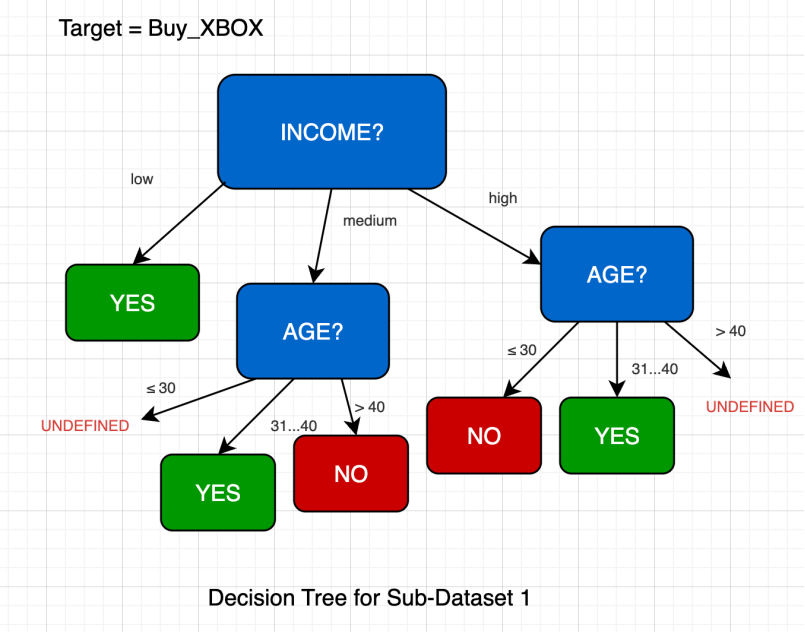
Processing branch: Income = low
→ Leaf node, stopping.

Processing branch: Income = medium
→ Going deeper using feature: Age

Subset: Age = 31...40
Student Credit_Rating Buy_XBOX
5 no Excellent yes
Pure subset

Subset: Age = >40
Student Credit_Rating Buy_XBOX
7 no Excellent no
Pure subset

Construct the decision tree for Subset 1:



Decision Tree for Sub-Dataset 1

Now, right at the first tree, we notice something interesting.

When we construct the Decision Tree using this sub-dataset, we follow the usual process: compute Information Gain, pick the best feature, and split the data accordingly. But because this is only a **subset** of the original dataset, something unexpected happens.

There are combinations of features that simply do not exist.

For example, when we split using **Age** and **Income**, we might realize that:

- There is **no record** of someone with *Income = high* and *Age > 40*
- There is also **no record** of someone with *Income = medium* and *Age ≤ 30*

These cases are not wrong—they are just **missing** in this particular subset due to random sampling. So the question becomes:

> What do we do when the tree encounters a situation it has never seen before?

The answer is simple—and very important. We assign the **majority class**.

In this subset, if we observe the labels carefully, we see that **"yes"** appears more frequently than "no". That means when the model reaches an undefined branch—where no training data exists—it defaults to the most common outcome it has learned so far.

So in this first tree, since **"yes" is the dominant class**, all undefined branches are assigned **"yes"**. With that:

> *"This tree does not know everything... it only knows what it has seen. And when it doesn't know, it follows the majority."*

And this is exactly where Random Forest starts to reveal its strength.

Because each tree is trained on a **different subset**, each one will have its own missing cases, its own biases, and its own majority decisions. One tree might lean toward "yes" in uncertain cases, while another might lean toward "no".

Individually, each tree is imperfect. But together, they compensate for each other. This “imperfection” is not a flaw. It is the very reason why the forest works.

Repeat the same for the other twos:

```
sub2 = {
    "Age": [">40", "≤30", "≤30", ">40", ">40", "≤30", "31…40", ">40",
"≤30", "≤30"],
    "Income": ["medium", "medium", "medium", "medium", "low", "medium",
"high", "medium", "high", "low"],
    "Student": ["no", "no", "no", "yes", "yes", "yes", "yes", "no", "n
o", "yes"],
    "Credit_Rating": ["Fair", "Fair", "Fair", "Fair", "Excellent", "Exc
ellent", "Fair", "Fair", "Excellent", "Fair"],
    "Buy_XBOX": ["yes", "no", "no", "yes", "no", "yes", "yes", "yes",
"no", "yes"]
}

X = pd.DataFrame(sub2)
y = X.pop("Buy_XBOX")
results, best_feature = information_gain(X, y)
# Iterate through the result dictionary
for feature, info in results.items():
      print(f"{feature}: {info}")
print("+ Best Feature:", best_feature, "\n")

next_best_features = recursive_split(X, y, best_feature)
all_next_levels = process_next_level(X, y, best_feature, next_best_feat
ures)
```

```
Age: {'information_gain': 0.1609640474436811}
Income: {'information_gain': 0.01997309402197489}
Student: {'information_gain': 0.12451124978365313}
Credit_Rating: {'information_gain': 0.0912774462416801}
+ Best Feature: Age

Subset: Age = >40
Income Student Credit_Rating Buy_XBOX
0  medium     no        Fair     yes
3  medium    yes        Fair     yes
4     low    yes   Excellent      no
7  medium     no        Fair     yes
Income: {'information_gain': 0.8112781244591328}
Student: {'information_gain': 0.31127812445913283}
Credit_Rating: {'information_gain': 0.8112781244591328}
 + Best Feature: Income

Subset: Age = ≤30
Income Student Credit_Rating Buy_XBOX
1  medium     no        Fair      no
2  medium     no        Fair      no
5  medium    yes   Excellent     yes
8    high     no   Excellent      no
9     low    yes        Fair     yes
```

Income: {'information_gain': 0.4199730940219749}
Student: {'information_gain': 0.9709505944546686}
Credit_Rating: {'information_gain': 0.01997309402197489}
+ Best Feature: Student

Subset: Age = 31...40
Income Student Credit_Rating Buy_XBOX
6 high yes Fair yes
Pure subset

Processing branch: Age = >40
→ Going deeper using feature: Income

Subset: Income = medium
Student Credit_Rating Buy_XBOX
0 no Fair yes
3 yes Fair yes
7 no Fair yes
Pure subset

Subset: Income = low
Student Credit_Rating Buy_XBOX
4 yes Excellent no
Pure subset

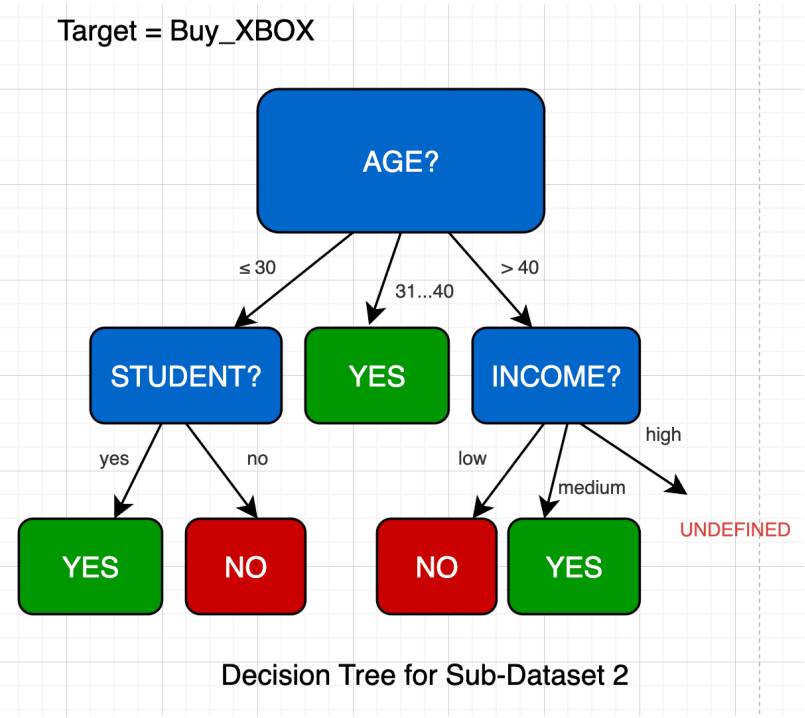
Processing branch: Age = ≤30
→ Going deeper using feature: Student

Subset: Student = no
Income Credit_Rating Buy_XBOX
1 medium Fair no
2 medium Fair no
8 high Excellent no
Pure subset

Subset: Student = yes
Income Credit_Rating Buy_XBOX
5 medium Excellent yes
9 low Fair yes
Pure subset

Processing branch: Age = 31...40
→ Leaf node, stopping.

Construct the decision tree for Subset 2:

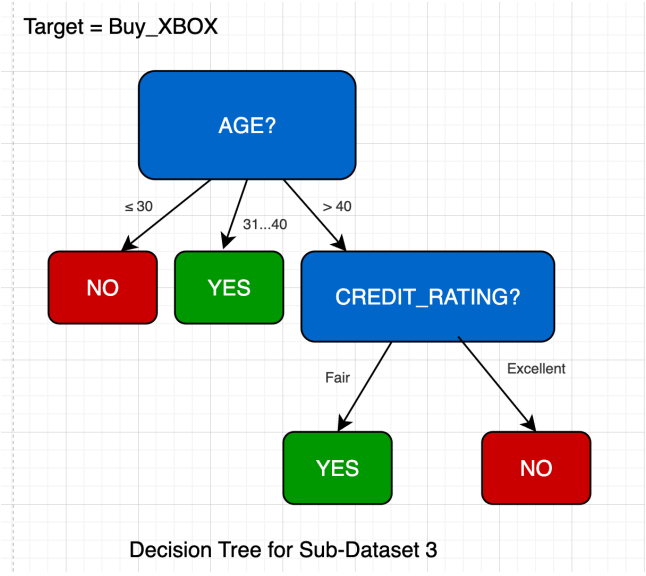

Target = Buy_XBOX
AGE?
≤ 30
31...40
> 40
STUDENT?
YES
INCOME?
yes
no
low
medium
high
UNDEFINED
YES
NO
NO
YES
Decision Tree for Sub-Dataset 2

```
sub3 = {
    "Age": ["≤30", "31…40", "31…40", ">40", ">40", ">40", "31…40", ">4
0", ">40", "31…40"],
    "Income": ["high", "high", "high", "low", "low", "low", "medium",
"medium", "medium", "low"],
    "Student": ["no", "no", "no", "yes", "yes", "yes", "no", "yes", "n
o", "yes"],
    "Credit_Rating": ["Fair", "Fair", "Fair", "Fair", "Excellent", "Exc
ellent", "Excellent", "Fair", "Excellent", "Excellent"],
    "Buy_XBOX": ["no", "yes", "yes", "yes", "no", "no", "yes", "yes",
"no", "yes"]
}

X = pd.DataFrame(sub3)
y = X.pop("Buy_XBOX")
results, best_feature = information_gain(X, y)
# Iterate through the result dictionary
for feature, info in results.items():
      print(f"{feature}: {info}")
print("+ Best Feature:", best_feature, "\n")

next_best_features = recursive_split(X, y, best_feature)
all_next_levels = process_next_level(X, y, best_feature, next_best_feat
ures)
```

```
Age: {'information_gain': 0.4854752972273343}
Income: {'information_gain': 0.01997309402197489}
Student: {'information_gain': 0.0}
Credit_Rating: {'information_gain': 0.12451124978365313}
 + Best Feature: Age

Subset: Age = ≤30
Income Student Credit_Rating Buy_XBOX
0   high     no        Fair      no
Pure subset

Subset: Age = 31...40
Income Student Credit_Rating Buy_XBOX
1   high     no        Fair     yes
2   high     no        Fair     yes
6 medium     no   Excellent     yes
9    low    yes   Excellent     yes
Pure subset

Subset: Age = >40
Income Student Credit_Rating Buy_XBOX
3    low    yes        Fair     yes
4    low    yes   Excellent      no
5    low    yes   Excellent      no
```

```
7  medium     yes         Fair      yes
8  medium      no     Excellent       no
Income: {'information_gain': 0.01997309402197489}
Student: {'information_gain': 0.17095059445466854}
Credit_Rating: {'information_gain': 0.9709505944546686}
+ Best Feature: Credit_Rating

Processing branch: Age = ≤30
→ Leaf node, stopping.

Processing branch: Age = 31...40
→ Leaf node, stopping.

Processing branch: Age = >40
→ Going deeper using feature: Credit_Rating

Subset: Credit_Rating = Fair
Income Student Buy_XBOX
3     low     yes      yes
7  medium     yes      yes
Pure subset

Subset: Credit_Rating = Excellent
Income Student Buy_XBOX
4     low     yes       no
5     low     yes       no
8  medium      no       no
Pure subset
```

Construct the decision tree for Subset 3:



Decision Tree for Sub-Dataset 3

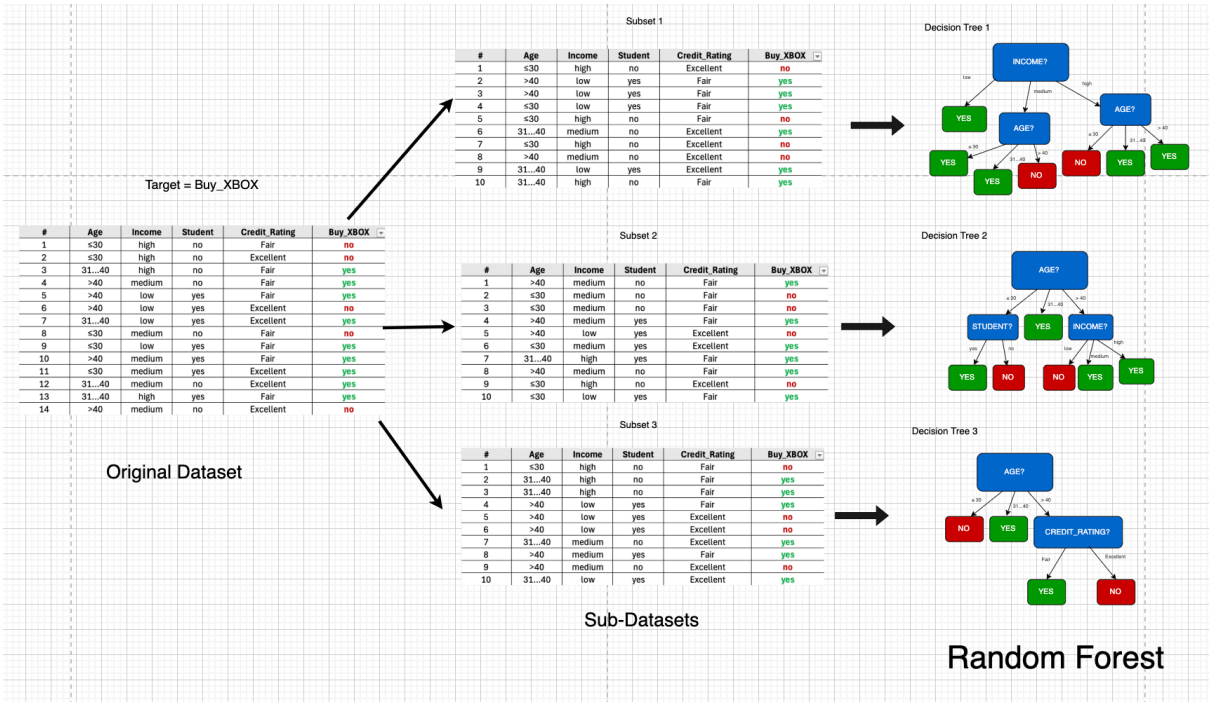
So when we say *"train the sub-datasets"*, it literally means:

- Take Subset 1 → run your Decision Tree algorithm → produce Tree 1

Then repeat:

- Take Subset 2 → run the same algorithm → produce Tree 2
- Take Subset 3 → run the same algorithm → produce Tree 3
- ... and so on, until we build the entire forest.

This image ideally encapsulates the entire idea: ***Multiple Decision Trees → "Random" Forest***




Each tree is trained in isolation, using only the data it was given. No sharing, no correction, no synchronization. And that's the key idea:

> Random Forest does not invent a new learning algorithm—it simply **reuses Decision Trees in a smarter way**, by controlling the data each tree learns from.

Once all trees are trained, we move to the final step: Combining their predictions—which is where the "forest" truly becomes more powerful than any single tree.

Now suppose we are given a new input:

| # | Age | Income | Student | Credit_Rating | Buy_XBOX |
|---|---|---|---|---|---|
| 15 | ≤30 | low | no | Fair | ? |

- Tree 1: **Income: low → Age ≤ 30 → Predict Yes**
- Tree 2: **Age ≤ 30 → Student: No → Predict No**
- Tree 3: **Age ≤ 30 → Predict No**

Final prediction: NO (based on major-voting)

At this point, the forest is already trained. Each tree has learned from its own subset, built its own structure, and developed its own way of making decisions.

So now, we let each tree do what it was trained to do—**predict independently**.

- **Tree 1: Income: low → Age ≤ 30 → Predict Yes**
- **Tree 2: Age ≤ 30 → Student: No → Predict No**
- **Tree 3: Age ≤ 30 → Predict No**

And now we pauses again: ***"Three trees... three perspectives... not identical, not perfect—but each one speaks."***

So how do we combine them? We use **majority voting**.

- Yes → 1 vote
- No → 2 votes

→ **Final Prediction:** $NO$

And this is the moment where the idea of a *forest* truly makes sense. A single tree might be wrong. But when multiple trees vote together, the final decision becomes more stable, more reliable.

Now, after understanding classification, we ask one more question:

**What if this is a regression problem instead?** The idea does not change—but the way we combine results does.

- Each tree becomes a **Regression Tree** (you've already seen this in the Decision Tree section)
- Instead of predicting a class like "yes" or "no", each tree predicts a **numerical value**
- And instead of voting, we take the **average (mean)** of all predictions

So: Classification → Majority Voting; and Regression → Mean (Average)

## 4. Random Forest with Python

Here's how you would build a Random Forest model in Python using the Scikit-Learn library:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import accuracy_score

# Load the dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='target')

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
2,
                                                    random_state=42)


# Train the model
# n_estimators is basically the number of decision trees (100 is defaul
t)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Optional: Evaluate
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9649122807017544
```

Before we move to the summary, there is one more important concept we should briefly touch on: **Feature Importance**.

Remember how in a single Decision Tree, we can clearly see which feature is the most important? That's because the tree explicitly chooses splits based on **Information Gain (IG)**, so the "best" feature is immediately visible at the top of the tree.

But Random Forest is different. We are no longer dealing with one tree—we are dealing with **many trees**, each trained on different subsets of data. And because of that, each tree might choose a *different* feature as its root, or prioritize features differently during splitting.

So now the question becomes:

> ***Across all these trees... which feature actually matters the most?***

That's where **Feature Importance** comes in.

In Python, using libraries like Scikit-Learn, we can extract this information directly:

```
# Feature Importance
importances = model.feature_importances_
feature_names = X.columns
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances}).set_index('Feature')
print(feature_importance_df.sort_values(by='Importance', ascending=False))
```

| Feature | Importance |
|---|---|
| worst area | 0.153892 |
| worst concave points | 0.144663 |
| mean concave points | 0.106210 |
| worst radius | 0.077987 |
| mean concavity | 0.068001 |
| worst perimeter | 0.067115 |
| mean perimeter | 0.053270 |
| mean radius | 0.048703 |
| mean area | 0.047555 |
| worst concavity | 0.031802 |
| area error | 0.022407 |
| worst texture | 0.021749 |
| worst compactness | 0.020266 |
| radius error | 0.020139 |
| mean compactness | 0.013944 |
| mean texture | 0.013591 |
| perimeter error | 0.011303 |
| worst smoothness | 0.010644 |
| worst symmetry | 0.010120 |
| concavity error | 0.009386 |
| mean smoothness | 0.007285 |
| fractal dimension error | 0.005321 |
| compactness error | 0.005253 |
| worst fractal dimension | 0.005210 |
| texture error | 0.004724 |
| smoothness error | 0.004271 |
| symmetry error | 0.004018 |
| mean fractal dimension | 0.003886 |
| mean symmetry | 0.003770 |
| concave points error | 0.003513 |

**So, what does this number actually mean?**

In simple terms, it tells us how much each feature contributes to the overall decision-making process **across the entire forest**.

How exactly are these values calculated? To be honest, we don't need to go too deep here.

But intuitively, it *does* relate to the same idea we've seen before—**Information Gain**. Features that frequently create good splits (i.e., reduce uncertainty) across many trees will naturally

receive higher importance scores. Each tree contributes a little piece of information, and the forest aggregates all of it into these final importance values.

And that's the key idea.

- We are no longer asking: **"Which feature is best?"**
- We are asking: ***"Across many different perspectives... which feature consistently matters?"***

And that shift in thinking is what makes Random Forest not just powerful for prediction—but also useful for **understanding data**.

> *"The exact formula? Not our focus. The insight it gives? That's what matters."*

## Summary

Random Forest is powerful because it changes *how we trust models*.

- Instead of relying on one Decision Tree, it uses **many trees**
- It reduces **overfitting**, because no single tree sees the full dataset
- It introduces **randomness**, making the model less predictable—but more robust
- And by combining multiple imperfect models, it produces a **stronger final decision**

> *"One tree tries to understand the data... A forest learns to agree"*

## References


https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/

https://www.ibm.com/think/topics/random-forest

https://developers.google.com/machine-learning/decision-forests/random-forests

https://www.youtube.com/watch?v=J4Wdy0Wc_xQ


### Next Section:


11. Gradient Boosting